

Laboratorium nr 3 – Administracja BD

System Zarządzania Kluczami Obcymi

Paweł Myszka
Nr albumu: 331720

10 listopada 2025

1 Cel laboratorium

Celem laboratorium było stworzenie systemu do zarządzania kluczami obcymi w bazach danych. System umożliwia:

- **Zapamiętywanie** stanu kluczy obcych w bazie danych
- **Usuwanie** wszystkich kluczy obcych (np. przed ładowaniem danych)
- **Odtwarzanie** kluczy obcych ze sprawdzeniem spójności danych
- **Raportowanie** błędów podczas odtwarzania

Typowy scenariusz użycia:

1. Przed załadowaniem dużej ilości danych – kasujemy klucze obce
2. Ładujemy dane bez przejmowania się kolejnością
3. Po załadowaniu – odtwarzamy klucze obce
4. System automatycznie wykrywa i raportuje problemy ze spójnością

2 Struktura systemu

2.1 Tabele administracyjne

2.1.1 FK_STORE – Nagłówek zapisu kluczy

```
1 CREATE TABLE dbo.FK_STORE (  
2     store_id INT IDENTITY(1,1) PRIMARY KEY,  
3     [desc] NVARCHAR(250) NOT NULL,  
4     db NVARCHAR(100) NOT NULL,  
5     start_dstamp DATETIME NOT NULL DEFAULT GETDATE(),  
6     end_dstamp DATETIME,  
7     err_msg NVARCHAR(500),  
8     rmv_after_store BIT DEFAULT 0  
9 );
```

Pole:

- **store_id** – ID operacji zapisu
- **desc** – Opis operacji
- **db** – Nazwa bazy danych

- start_dstamp – Data rozpoczęcia
- end_dstamp – Data zakończenia (NULL = nie zakończono)
- err_msg – Komunikat błędu (NULL = OK)

2.1.2 FK_STORE_DET – Szczegóły zapisanych kluczy

```
1 CREATE TABLE dbo.FK_STORE_DET (  
2     det_id INT IDENTITY(1,1) PRIMARY KEY,  
3     store_id INT NOT NULL,  
4     master_tab NVARCHAR(100) NOT NULL,  
5     master_col NVARCHAR(100) NOT NULL,  
6     details_tab NVARCHAR(100) NOT NULL,  
7     details_col NVARCHAR(100) NOT NULL,  
8     fk_name NVARCHAR(250) NOT NULL,  
9     removed BIT DEFAULT 0,  
10    FOREIGN KEY (store_id) REFERENCES FK_STORE(store_id)  
11 );
```

Pola:

- det_id – ID szczegółu
- store_id – Powiązanie z FK_STORE
- master_tab, master_col – Tabela i kolumna master
- details_tab, details_col – Tabela i kolumna details
- fk_name – Nazwa klucza obcego
- removed – Czy klucz został usunięty (1/0)

2.1.3 FK_RESTORE – Nagłówek odtwarzania

```
1 CREATE TABLE dbo.FK_RESTORE (  
2     restore_id INT IDENTITY(1,1) PRIMARY KEY,  
3     [desc] NVARCHAR(250) NOT NULL,  
4     db NVARCHAR(100) NOT NULL,  
5     start_dstamp DATETIME NOT NULL DEFAULT GETDATE(),  
6     end_dstamp DATETIME,  
7     err_msg NVARCHAR(500)  
8 );
```

2.1.4 FK_RESTORE_JOB – Szczegóły odtwarzania

```
1 CREATE TABLE dbo.FK_RESTORE_JOB (  
2     job_id INT IDENTITY(1,1) PRIMARY KEY,  
3     package_id INT NOT NULL,  
4     detail_col NVARCHAR(100),  
5     detail_table NVARCHAR(100),  
6     master_col NVARCHAR(100),  
7     master_table NVARCHAR(100),  
8     db NVARCHAR(100),  
9     fk_name NVARCHAR(250),  
10    fk_already_exists BIT,  
11    data_inconsistency BIT,  
12    cr_start DATETIME,  
13    cr_end DATETIME,
```

```

14     err_msg NVARCHAR(500),
15     FOREIGN KEY (package_id) REFERENCES FK_RESTORE(restore_id)
16 );

```

2.1.5 FK_CHECK_QUERY – Cache zapytań

```

1 CREATE TABLE dbo.FK_CHECK_QUERY (
2     check_id INT IDENTITY(1,1) PRIMARY KEY,
3     col_name_where_data_exists NVARCHAR(100),
4     tab_where_data_exists NVARCHAR(100),
5     col_to_check NVARCHAR(100),
6     tab_to_check NVARCHAR(100),
7     db NVARCHAR(100),
8     s_query_check NVARCHAR(MAX),
9     s_query_show NVARCHAR(MAX),
10    UNIQUE (db, tab_to_check, col_to_check, tab_where_data_exists,
11            col_name_where_data_exists)
12 );

```

2.2 Funkcje pomocnicze

2.2.1 F_REF_CHECK_QUERY

Generuje zapytanie SQL sprawdzające czy istnieją rekordy w tabeli details, które nie mają odpowiednika w tabeli master:

```

1 CREATE FUNCTION dbo.F_REF_CHECK_QUERY(
2     @details_col NVARCHAR(100),
3     @details_tab NVARCHAR(100),
4     @master_col NVARCHAR(100),
5     @master_tab NVARCHAR(100),
6     @db NVARCHAR(100)
7 )
8 RETURNS NVARCHAR(2000)
9 AS
10 BEGIN
11     DECLARE @result NVARCHAR(2000);
12     SET @result = N'USE [' + @db + N']; SET @e=0; '
13         + N'IF EXISTS (SELECT 1 FROM [' + @details_tab + N']) '
14         + N'WHERE NOT EXISTS (SELECT 1 FROM [' + @master_tab + N']) '
15         + N'WHERE [' + @details_tab + N'].[' + @details_col + N'] '
16         + N'=[ ' + @master_tab + N'].[' + @master_col + N']) '
17         + N'SET @e=1';
18     RETURN @result;
19 END

```

2.2.2 F_REF_RESULTS

Generuje zapytanie SQL pokazujące niespójne dane:

```

1 CREATE FUNCTION dbo.F_REF_RESULTS(
2     @details_col NVARCHAR(100),
3     @details_tab NVARCHAR(100),
4     @master_col NVARCHAR(100),
5     @master_tab NVARCHAR(100),
6     @db NVARCHAR(100)
7 )
8 RETURNS NVARCHAR(2000)
9 AS
10 BEGIN

```

```

11 DECLARE @result NVARCHAR(2000);
12 SET @result = N'USE_' + @db + N';'
13 + N'SELECT DISTINCT_' + @details_tab + N'].[' + @details_col +
    N']_'
14 + N'AS_' + @details_col + N'_w_' + @details_tab
15 + N'_nie_ma_w_' + @master_tab + N']_'
16 + N'FROM_' + @details_tab + N']_'
17 + N'WHERE NOT EXISTS (SELECT 1 FROM_' + @master_tab + N']_'
18 + N'WHERE_' + @details_tab + N'].[' + @details_col + N']_'
19 + N'=[_' + @master_tab + N'].[' + @master_col + N'])';
20 RETURN @result;
21 END

```

3 Procedury zapamietywania i usuwania

3.1 Procedura z3_fk_store

Zapamiętuje wszystkie klucze obce z danej bazy:

```

1 CREATE PROCEDURE z3_fk_store
2     @db nvarchar(100),
3     @store_id int = NULL OUTPUT
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7     DECLARE @sql nvarchar(max), @err_msg nvarchar(250);
8
9     BEGIN TRY
10         -- Wstawiamy rekord do FK_STORE
11         INSERT INTO FK_STORE ([desc], db, rmv_after_store)
12         VALUES ('Save_FK_for_database_' + @db, @db, 0);
13
14         SET @store_id = SCOPE_IDENTITY();
15
16         -- Pobieramy klucze obce z danej bazy
17         SET @sql = N'USE_' + QUOTENAME(@db) + N';
18         INSERT INTO B25_ADM_PM331720.dbo.FK_STORE_DET
19         (store_id, master_tab, master_col,
20         details_tab, details_col, fk_name, removed)
21         SELECT_' + CAST(@store_id AS nvarchar(20)) + N',
22         OBJECT_NAME(f.referenced_object_id),
23         COL_NAME(fc.referenced_object_id, fc.
24         referenced_column_id),
25         OBJECT_NAME(f.parent_object_id),
26         COL_NAME(fc.parent_object_id, fc.parent_column_id),
27         f.name, 0
28         FROM sys.foreign_keys AS f
29         JOIN sys.foreign_key_columns AS fc
30         ON f.[object_id] = fc.constraint_object_id
31         ORDER BY f.name';
32
33         EXEC sp_executesql @sql;
34
35         UPDATE FK_STORE
36         SET end_dstamp = GETDATE()
37         WHERE store_id = @store_id;
38
39         PRINT 'Saved_FK_for_database_' + @db
40             + '_(store_id=' + CAST(@store_id AS varchar(10)) + ')';
41     END TRY
42     BEGIN CATCH

```

```

43         SET @err_msg = ERROR_MESSAGE();
44         UPDATE FK_STORE
45         SET err_msg = @err_msg
46         WHERE store_id = @store_id;
47         PRINT 'ERROR_during_save:' + @err_msg;
48     END CATCH
49 END

```

Logika:

1. Wstawia rekord nagłówka do FK_STORE
2. Pobiera store_id
3. Buduje dynamiczne SQL do pobrania FK z danej bazy
4. Wstawia szczegóły do FK_STORE_DET
5. Ustawia date zakończenia

3.2 Procedura z3_fk_rmv

Usuwa wszystkie klucze obce z danej bazy (najpierw je zapamiętuje!):

```

1  CREATE PROCEDURE z3_fk_rmv
2      @db nvarchar(100)
3  AS
4  BEGIN
5      SET NOCOUNT ON;
6      DECLARE @store_id int, @det_id int, @fk_name nvarchar(250),
7              @details_tab nvarchar(100), @sql nvarchar(max),
8              @err int, @err_msg nvarchar(250);
9
10     BEGIN TRY
11         -- Najpierw zapami tujemy klucze
12         EXEC z3_fk_store @db = @db, @store_id = @store_id OUTPUT;
13
14         PRINT 'Starting_FK_removal_for_database' + @db
15             + '_(store_id=' + CAST(@store_id AS varchar(10)) + ')';
16
17         -- Cursor po zapisanych kluczach
18         DECLARE fk_cursor CURSOR FOR
19             SELECT det_id, fk_name, details_tab
20             FROM FK_STORE_DET
21             WHERE store_id = @store_id ORDER BY det_id;
22
23         OPEN fk_cursor;
24         FETCH NEXT FROM fk_cursor INTO @det_id, @fk_name, @details_tab;
25
26         WHILE @@FETCH_STATUS = 0
27             BEGIN
28                 BEGIN TRY
29                     -- Budujemy polecenie usuni cia
30                     SET @sql = N'USE' + QUOTENAME(@db)
31                         + N';ALTER_TABLE' + QUOTENAME(
32                             @details_tab)
33                         + N'_DROP_CONSTRAINT' + QUOTENAME(
34                             @fk_name);
35
36                     EXEC sp_executesql @sql;
37                     SET @err = @@ERROR;
38
39                     IF @err = 0

```

```

38         BEGIN
39             UPDATE FK_STORE_DET
40             SET removed = 1
41             WHERE det_id = @det_id;
42             PRINT 'Removed_FK:' + @fk_name
43                 + 'from table' + @details_tab
44                 ;
45         END
46     END TRY
47     BEGIN CATCH
48         SET @err_msg = ERROR_MESSAGE();
49         PRINT 'ERROR_removing_FK' + @fk_name
50             + ': ' + @err_msg;
51     END CATCH
52
53     FETCH NEXT FROM fk_cursor
54     INTO @det_id, @fk_name, @details_tab;
55
56     END
57
58     CLOSE fk_cursor;
59     DEALLOCATE fk_cursor;
60
61     PRINT 'Finished_FK_removal_for_database' + @db;
62
63     END TRY
64     BEGIN CATCH
65         IF CURSOR_STATUS('local', 'fk_cursor') >= 0
66         BEGIN
67             CLOSE fk_cursor;
68             DEALLOCATE fk_cursor;
69         END
70         SET @err_msg = ERROR_MESSAGE();
71         PRINT 'CRITICAL_ERROR:' + @err_msg;
72     END CATCH
73 END

```

Logika:

1. Najpierw zapamiętuje klucze procedura z3_fk_store
2. Otwiera kursor po zapisanych kluczach
3. Dla każdego klucza – buduje SQL DROP CONSTRAINT
4. Oznacza jako usunięty w FK_STORE_DET
5. Obsługa błędów przy usuwaniu (TRY-CATCH)

4 Procedury odtwarzania

4.1 Procedura z3_fk_restore

Odtwarza klucze obce ze sprawdzeniem spójności danych:

```

1 CREATE PROCEDURE z3_fk_restore
2     @db nvarchar(100)
3 AS
4 BEGIN
5     SET NOCOUNT ON;
6     DECLARE @store_id int, @restore_id int, @master_tab nvarchar(100),
7             @master_col nvarchar(100), @details_tab nvarchar(100),
8             @details_col nvarchar(100), @fk_name nvarchar(250),

```

```

9      @sql nvarchar(max), @check_sql nvarchar(2000),
10     @show_sql nvarchar(2000), @err int,
11     @data_inconsistency bit, @fk_already_exists bit,
12     @job_id int, @err_msg nvarchar(250);
13
14 BEGIN TRY
15     -- Szukamy ostatniego udanego store dla bazy
16     SELECT TOP 1 @store_id = store_id
17     FROM FK_STORE
18     WHERE db = @db AND end_dstamp IS NOT NULL
19           AND err_msg IS NULL
20     ORDER BY end_dstamp DESC;
21
22     IF @store_id IS NULL
23     BEGIN
24         PRINT 'No saved FK state found for database' + @db;
25         RETURN;
26     END
27
28     PRINT 'Found store_id=' + CAST(@store_id AS varchar(10))
29         + ' for database' + @db;
30
31     -- Tworzymy rekord w FK_RESTORE
32     INSERT INTO FK_RESTORE ([desc], db)
33     VALUES ('Restore FK for database' + @db, @db);
34
35     SET @restore_id = SCOPE_IDENTITY();
36     PRINT 'Created restore_id=' + CAST(@restore_id AS varchar(10));
37
38     -- Kursor po kluczach do odtworzenia
39     DECLARE restore_cursor CURSOR FOR
40         SELECT master_tab, master_col, details_tab,
41               details_col, fk_name
42         FROM FK_STORE_DET
43         WHERE store_id = @store_id
44         ORDER BY det_id;
45
46     OPEN restore_cursor;
47     FETCH NEXT FROM restore_cursor
48         INTO @master_tab, @master_col,
49             @details_tab, @details_col, @fk_name;
50
51     WHILE @@FETCH_STATUS = 0
52     BEGIN
53         BEGIN TRY
54             SET @data_inconsistency = 0;
55             SET @fk_already_exists = 0;
56             SET @err_msg = NULL;
57
58             -- Sprawdzamy czy zapytanie jest w cache
59             IF NOT EXISTS (
60                 SELECT 1 FROM FK_CHECK_QUERY
61                 WHERE db = @db
62                       AND tab_to_check = @master_tab
63                       AND col_to_check = @master_col
64                       AND tab_where_data_exists =
65                           @details_tab
66                       AND col_name_where_data_exists
67                           = @details_col
68             )
69             BEGIN
70                 SET @check_sql = dbo.F_REF_CHECK_QUERY(
71                     @details_col, @details_tab,

```

```

70         @master_col, @master_tab, @db);
71     SET @show_sql = dbo.F_REF_RESULTS(
72         @details_col, @details_tab,
73         @master_col, @master_tab, @db);
74
75     INSERT INTO FK_CHECK_QUERY
76         (col_name_where_data_exists,
77         tab_where_data_exists,
78         col_to_check,
79         tab_to_check, db, s_query_check
80         ,
81         s_query_show)
82     VALUES (@details_col, @details_tab,
83         @master_col, @master_tab, @db,
84         @check_sql, @show_sql);
85
86     END
87     ELSE
88     BEGIN
89         SELECT @check_sql = s_query_check,
90             @show_sql = s_query_show
91         FROM FK_CHECK_QUERY
92         WHERE db = @db
93             AND tab_to_check = @master_tab
94             AND col_to_check = @master_col
95             AND tab_where_data_exists =
96                 @details_tab
97             AND col_name_where_data_exists
98                 = @details_col;
99
100    END
101
102    -- Sprawdzamy sp jno danych
103    DECLARE @data_err int = 0;
104    EXEC sp_executesql @check_sql,
105        N'@e int OUTPUT',
106        @e = @data_err OUTPUT;
107
108    IF @data_err = 1
109        SET @data_inconsistency = 1;
110
111    -- Sprawdzamy czy klucz ju istnieje
112    SET @sql = N'USE ' + QUOTENAME(@db) + N';
113    SELECT @exists = COUNT(*)
114    FROM sys.foreign_keys AS f
115    JOIN sys.foreign_key_columns AS fc
116    ON f.[object_id] = fc.
117        constraint_object_id
118    WHERE OBJECT_NAME(f.parent_object_id) =
119        @det_tab
120    AND COL_NAME(fc.parent_object_id, fc.
121        parent_column_id) = @det_col
122    AND OBJECT_NAME(f.referenced_object_id)
123        = @mst_tab
124    AND COL_NAME(fc.referenced_object_id,
125        fc.referenced_column_id) = @mst_col';
126
127    DECLARE @exists int = 0;
128    EXEC sp_executesql @sql,
129        N'@det_tab nvarchar(100), @det_col
130        nvarchar(100),
131        @mst_tab nvarchar(100), @mst_col
132        nvarchar(100),
133        @exists int OUTPUT',
134        @det_tab = @details_tab, @det_col =

```



```

122         @details_col,
123         @mst_tab = @master_tab, @mst_col =
124             @master_col,
125         @exists = @exists OUTPUT;
126
127 IF @exists > 0
128     SET @fk_already_exists = 1;
129
130 -- Wstawiamy job
131 INSERT INTO FK_RESTORE_JOB
132     (package_id, detail_col, detail_table,
133     master_col, master_table, db, fk_name,
134     fk_already_exists, data_inconsistency)
135 VALUES (@restore_id, @details_col, @details_tab
136     ,
137     @master_col, @master_tab, @db, @fk_name
138     ,
139     @fk_already_exists, @data_inconsistency
140     );
141
142 SET @job_id = SCOPE_IDENTITY();
143
144 -- Tworzymy klucz tylko bez problem w
145 IF @fk_already_exists = 0 AND
146     @data_inconsistency = 0
147 BEGIN
148     SET @sql = N'USE_' + QUOTENAME(@db)
149         + N';ALTER_' + QUOTENAME
150             (@details_tab)
151         + N'_ADD_' + QUOTENAME
152             (@fk_name)
153         + N'_FOREIGN_' + QUOTENAME
154             (@details_col) + N')_'
155         + N'REFERENCES_' + QUOTENAME(
156             @master_tab)
157         + N'(' + QUOTENAME(@master_col)
158             + N')';
159
160 EXEC sp_executesql @sql;
161
162 UPDATE FK_RESTORE_JOB
163 SET cr_end = GETDATE()
164 WHERE job_id = @job_id;
165
166 PRINT 'Created_' + @fk_name;
167
168 END
169 ELSE
170 BEGIN
171     IF @fk_already_exists = 1
172         SET @err_msg = 'FK_' + @fk_name +
173             'already exists';
174     IF @data_inconsistency = 1
175         SET @err_msg = ISNULL(@err_msg,
176             + 'Data inconsistency');
177
178 UPDATE FK_RESTORE_JOB
179 SET err_msg = @err_msg
180 WHERE job_id = @job_id;
181
182 PRINT 'Skipped_' + @fk_name + ': ' +
183     @err_msg;
184
185 END

```

```
171         END TRY
172         BEGIN CATCH
173             SET @err_msg = ERROR_MESSAGE();
174             UPDATE FK_RESTORE_JOB
175             SET err_msg = @err_msg
176             WHERE job_id = @job_id;
177             PRINT 'ERROR_creating_FK' + @fk_name
178                 + ': ' + @err_msg;
179         END CATCH
180
181         FETCH NEXT FROM restore_cursor
182         INTO @master_tab, @master_col,
183             @details_tab, @details_col, @fk_name;
184     END
185
186     CLOSE restore_cursor;
187     DEALLOCATE restore_cursor;
188
189     -- Sprawdzamy b   dy i raportujemy
190     IF EXISTS (SELECT 1 FROM FK_RESTORE_JOB
191               WHERE package_id = @restore_id
192               AND (err_msg IS NOT NULL OR fk_already_exists = 1
193                   OR data_inconsistency = 1))
194     BEGIN
195         PRINT 'Errors_detected_running_report...';
196         EXEC z3_fk_restore_report_errors
197             @restore_id = @restore_id;
198     END
199     ELSE
200     BEGIN
201         UPDATE FK_RESTORE
202         SET end_dstamp = GETDATE()
203         WHERE restore_id = @restore_id;
204         PRINT 'Successfully_restored_FK';
205     END
206
207 END TRY
208 BEGIN CATCH
209     IF CURSOR_STATUS('local', 'restore_cursor') >= 0
210     BEGIN
211         CLOSE restore_cursor;
212         DEALLOCATE restore_cursor;
213     END
214     SET @err_msg = ERROR_MESSAGE();
215     UPDATE FK_RESTORE
216     SET err_msg = @err_msg
217     WHERE restore_id = @restore_id;
218     PRINT 'CRITICAL_ERROR: ' + @err_msg;
219 END CATCH
220
221 END
```

5 Testowanie systemu

5.1 Przebieg testów

Testy zostały wykonane na bazie PM_DB1 z następującym przebiegiem:

5.1.1 TEST 1 – Zapis kluczy obcych

Polecenie:

```
1 DECLARE @store_id1 int;  
2 EXEC z3_fk_store @db = 'PM_DB1', @store_id = @store_id1 OUTPUT;
```

Wynik:

```
1 Saved FK for database PM_DB1 (store_id=4)
```

Zapisano 3 klucze obce z bazy PM_DB1:

- FK_ETATY_OSOBY_PM (ETATY.id_osoby → OSOBY.id_osoby)
- FK_MIASTA_WOJ_PM (MIASTA.kod_woj → WOJ.kod_woj)
- FK_OSOBY_MIASTA_PM (OSOBY.id_miasta → MIASTA.id_miasta)

5.1.2 TEST 2 – Sprawdzenie kluczy przed usunięciem

Zapytanie do widoku systemowego wykazało 3 klucze obce w bazie PM_DB1.

5.1.3 TEST 3 – Usunięcie kluczy obcych**Polecenie:**

```
1 EXEC z3_fk_rmv @db = 'PM_DB1';
```

Wyniki:

```
1 Starting FK removal for database PM_DB1 (store_id=5)  
2 Removed FK: FK_ETATY_OSOBY_PM from table ETATY  
3 Removed FK: FK_MIASTA_WOJ_PM from table MIASTA  
4 Removed FK: FK_OSOBY_MIASTA_PM from table OSOBY  
5 Finished FK removal for database PM_DB1
```

Po usunięciu weryfikacja wykazała brak kluczy obcych w bazie PM_DB1.

5.1.4 TEST 4 – Odtworzenie kluczy**Polecenie:**

```
1 EXEC z3_fk_restore @db = 'PM_DB1';
```

Wyniki:

```
1 Found store_id=5 for database PM_DB1  
2 Created restore_id=1  
3 Created FK: FK_MIASTA_WOJ_PM  
4 Created FK: FK_OSOBY_MIASTA_PM  
5 Skipped FK FK_ETATY_OSOBY_PM: Data inconsistency  
6 Errors detected - running report...
```

5.1.5 TEST 5 – Raport błędów**Polecenie:**

```
1 EXEC z3_fk_restore_report_errors @restore_id = 1;
```

Wyniki:

```

1  ERROR REPORT FOR restore_id = 1
2  =====
3
4  --- DATA INCONSISTENCY (details below) ---
5
6  >>> FK: FK_ETATY_OSOBY_PM
7      (ETATY -> OSOBY)
8      Inconsistent data:
9
10 id_osoby w OSOBY nie ma w ETATY
11 -----
12                               1
13
14 SUMMARY:
15 Total FK: 3
16 Successfully created: 2
17 Errors/Skipped: 1

```

System wykrył, że w tabeli ETATY brakuje referencji dla id_osoby=1, co uniemożliwiło odtworzenie klucza FK_ETATY_OSOBY_PM.

5.1.6 TEST 6 – Weryfikacja odtworzonych kluczy

Zapytanie do widoku systemowego wykazało 2 klucze obce (FK_MIASTA_WOJ_PM i FK_OSOBY_MIASTA_PM) trzeci został pominięty ze względu na niespójność danych.

6 Statystyka wyników

6.1 Podsumowanie testów

Poniższa tabela przedstawia rezultaty procedury z3_fk_restore:

Metrika	Wartość
Łącznie kluczy	3
Pomyślnie utworzonych	2
Już istnieją	0
Niespójne dane	1
Inne błędy	0

7 Wnioski

7.1 Funkcjonalność systemu

System zarządzania kluczami obcymi działa prawidłowo:

1. **Zapamiętywanie** – procedura z3_fk_store poprawnie identyfikuje i zapisuje wszystkie klucze obce z wybranej bazy
2. **Usuwanie** – procedura z3_fk_rmv bezpiecznie usuwa klucze (najpierw je zapamiętuje) i zapisuje status w tabeli
3. **Odtwarzanie** – procedura z3_fk_restore:
 - Sprawdza spójność danych przed utworzeniem klucza
 - Tworzy klucze tylko gdy dane są spójne
 - Pomija klucze już istniejące w bazie

- Generuje szczegółowy raport błędów
4. **Raportowanie** – procedura `z3_fk_restore_report_errors` wyświetla dokładne informacje o przyczynach niepowodzenia

7.2 Korzyści dla administratora

1. **Bezpieczne ładowanie danych** – możliwość wyłączenia FK przed bulkem load'em
2. **Automatyczne sprawdzenie spójności** – system nie pozwala na utworzenie FK z niespójnymi danymi
3. **Audyt operacji** – pełna historia zapisu, usuwania i odtwarzania FK
4. **Efektywność** – cache zapytań sprawdzających (FK_CHECK_QUERY) unika powtarzających się obliczeń

7.3 Możliwe rozszerzenia

- Automatyzacja procedur za pomocą agenta SQL Server
- Integracja z procesem ETL/ELT
- Rozszerzenie raportowania o statystyki szczegółowe
- Obsługa kluczy złożonych (multi-column FK)

8 Podsumowanie

W ramach laboratorium nr 3 zrealizowano kompleksowy system do zarządzania kluczami obcymi w bazach danych SQL Server. System umożliwia bezpieczne zapamiętywanie, usuwanie i odtwarzanie kluczy obcych ze sprawdzeniem spójności danych. Testowanie wykazało prawidłowe działanie wszystkich komponentów systemu.

*Paweł Myszka, nr albumu 331720
10 listopada 2025*